Software Engineering – Assignment 1

Project Definition, Development Approach Comparison and Project Plan

---

Proposed Project: Shared Digital Book Library

Team size: two members | Two short development iterations | Existing open-source codebase extended with new features

Task 1. Define the Project

1.1 Problem and Intended Value

The project extends a small open-source bookmark web application into a Shared Digital Book Library: a lightweight tool in which a small group keeps one shared catalogue, sees available stock, borrows and returns books, and joins a reservation queue when a copy is out. Its value is a simple, low-maintenance shared book list without the complexity of a full commercial library system.

1.2 Stakeholders

Stakeholder	Main role / interest
Library members (end users)	Browse and search; borrow, return and reserve books.
Administrator (team member)	Add / update book records, manage stock and the reservation queue.
Our two-person development team	Extend the codebase, implement conflict control, test and maintain the repository.
Lecturer / assessor	Evaluates scoping, approach choice, plan quality and teamwork evidence.
Upstream / future maintainers	Own the base open-source code; depend on MIT attribution and readable, documented code.

1.3 Scope, Exclusions and Constraints

In scope:

- Shared catalogue (title, author, copies and available stock); search and filter; borrow / return; reservation queue; conflict control; Git collaboration.

Out of scope:

- No payment or fines; no e-book online reading; no hardware (RFID/barcode) integration; no native mobile app.

Key constraints:

- Time: two short iterations. Team: two developers, so features stay minimal and prioritised.
- Base: extend an existing MIT-licensed open-source codebase and cite it; the race-condition conflict solution must be demonstrated; storage remains lightweight.

1.4 Relevant Software Engineering Issues

Issue 1 – Concurrent conflict (core). When only one copy remains, two simultaneous borrow requests could both be accepted and make stock inconsistent; two simultaneous edits could overwrite each other. We use version-based optimistic locking to block the losing request.

Issue 2 – Scope creep. New feature ideas must be filtered through a prioritised backlog, otherwise the two-person team cannot finish the core deliverables in two iterations.

Issue 3 – Team integration conflicts. Two developers can create merge conflicts; this is managed with feature branches, small commits and pull-request review.

1.5 Existing Code Acknowledgement

The project extends the open-source repository bookmarker by Brad Traversy (https://github.com/bradtraversy/bookmarker), MIT License. The original code provides the basic add / delete entry flow; library features (stock, borrow / return, reservation queue, conflict control, search) are added by our team.

Task 2. Compare and Select Development Approaches

2.1 Two Approaches Compared

Plan-driven (Waterfall): requirements, design, implementation and testing proceed in fixed sequence and requirements are frozen early. It gives clear documentation and a fixed scope, but is inflexible, tests late, and makes conflict logic expensive to fix.

Iterative / Incremental (Agile): the system is built in short iterations, each delivering a usable increment followed by review and backlog reprioritisation. It supports early testing and quick adjustment, but requires disciplined backlog control to avoid scope creep.

Dimension	Plan-driven (Waterfall)	Iterative / Incremental (Agile)
Requirements	Frozen before coding.	Refined each iteration.
Testing	Late, after implementation.	Continuous; conflict cases tested early.
Flexibility	Low; changes costly.	Higher; backlog reprioritised.
Team fit	Stable, regulated requirements.	Small team, exploratory features.
Conflict handling	Race conditions found late.	Built and validated in Iteration 1.

2.2 Selection and Context Differences

We select the iterative and incremental (agile) approach: it matches our two planned iterations, small team, and the need to build and test the conflict-control logic early. Iteration 1 delivers the catalogue, borrow / return and conflict backbone; Iteration 2 adds search, the reservation queue and final tests.

- In a regulated public-library context we would lean plan-driven, because compliance documents and fixed rules must be approved before coding.
- For a distributed team we would keep agile but add stricter pull-request review, written iteration notes and automated tests.

Task 3. Prepare the Project Plan

3.1 System Overview and Main Design Decisions

Read from left to right, the figure shows the request path: all users share one frontend, the version checker decides whether a change may proceed, and only verified changes reach the shared data store.

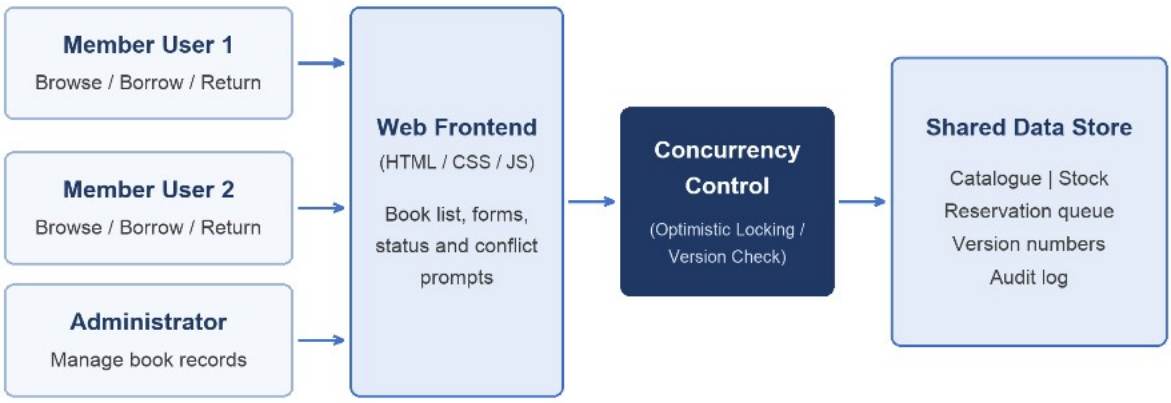


Figure 1. High-level system overview: users interact with the frontend; the version checker guards the shared data store.

Main design decisions:

- **Optimistic concurrency control with version numbers.** Every book record carries a version number. On borrow, return or edit, the request is accepted only if the supplied version matches the current stored version; otherwise the request is rejected and the user is asked to refresh. This prevents two requests consuming the last copy or two edits overwriting each other.
- **Single shared data model.** All members read and write the same catalogue state, unlike the original local browser storage; stock and reservations are computed from one source of truth.
- **Reservation queue on stock-out.** When available copies reach zero, new requesters join a FIFO queue instead of receiving an invalid borrow; a return releases the next queue member.
- **Branch-per-feature Git workflow.** The shared master / main branch stays stable; each feature (catalogue, conflict checker, reservation, search) lives on its own branch and merges through a reviewed pull request.
- **Core-first prioritisation.** Conflict control and correct stock transitions are prioritised over cosmetic features, because they define the project's engineering value.

3.2 Prioritised Product Backlog with Testable Acceptance Criteria

Pri.	Backlog Item	Testable Acceptance Criteria
High	BL-1 Shared book catalogue (add / list records)	Members see one common list; submitting title, author and copy count saves the record; all users reload and see the same entry.
High	BL-2 Stock-aware borrow and return	Borrow decreases available stock by one and marks the book as borrowed; return increases stock; stock never becomes negative.
High	BL-3 Optimistic conflict control (last-copy race condition)	Two simultaneous borrow requests for a book with one copy: exactly one succeeds; the other receives a clear conflict message and stock remains consistent; record version increments on each accepted change.

Pri.	Backlog Item	Testable Acceptance Criteria
Medium	BL-4 Concurrent edit conflict detection	When two users open the same record and the second submitter holds a stale version, the save is blocked and the record must be refreshed before re-submission.
Medium	BL-5 Reservation queue	When available stock is zero, a borrow request adds the user to the FIFO queue; on return, the first queued member is notified / marked reserved; queue order cannot be skipped.
Medium	BL-6 Search and filter	Users can filter by title keyword, author, and status (available / borrowed); results update immediately and show correct stock.
Low	BL-7 Simple audit log	Every borrow, return and accepted edit records user action, timestamp and affected book; the log can be listed for review.
Low	BL-8 Per-book note field	A member can attach a short note to a book; saving the note respects the version check from BL-4.

3.3 Team Responsibilities (Two Members)

Member	Primary Responsibilities	Shared / Review Responsibilities
Bao Nanxi	Data model and stock state; BL-2 borrow / return; BL-3 optimistic conflict control and version logic; conflict test cases; system diagram and plan documentation.	Iteration review; pull-request review of Zhou Shuyue's branches; risk tracking; final report assembly.
Zhou Shuyue	Frontend catalogue UI and forms; BL-1 record entry; BL-4 edit conflict prompts; BL-6 search / filter; Git repository setup and workflow evidence (issues, branches, PRs, commits).	Iteration review; pull-request review of Bao Nanxi's branches; acceptance-criteria checks; preparation of submission evidence.

3.4 Goals for the Two Short Iterations

Iteration 1 – Core Working System (foundation + conflict backbone)

- **Goal:** a working shared catalogue in which stock and borrow / return behave correctly under a simultaneous-request conflict.
- **Backlog:** BL-1, BL-2, BL-3; plus initial trial of the branch / issue / pull-request workflow.
- **Deliverable:** book list and add form; borrow and return; version check that rejects a losing concurrent borrow request; a manual conflict test record; the shared repository skeleton.
- **Exit criterion:** stock never goes negative; exactly one request wins when one copy remains; basic workflow evidence (issue, feature branch, PR, merge) exists.

Iteration 2 – Enhancement, Validation and Submission Readiness

- **Goal:** add medium-value features, harden error handling, complete testing and assemble assignment evidence.
- **Backlog:** BL-4, BL-5, BL-6; optionally BL-7 / BL-8 if capacity allows after high-priority items are complete.
- **Deliverable:** stale-edit conflict prompt; reservation queue; search and filter; consolidated test results; risk register updated; contribution record with links to issues, commits and PRs.
- **Exit criterion:** every high-priority acceptance criterion passes; medium features meet their criteria; the workflow trial and contributions are documented for submission.

3.5 Key Risks and Management

Risk	Impact	Mitigation Strategy
Race-condition logic contains a hidden bug	High	Write dedicated test cases for two concurrent borrows and stale edits; test the conflict path manually in Iteration 1 before adding features.
Scope creep from new feature ideas	Medium	Keep a fixed prioritised backlog; low-priority items enter an iteration only after high-priority items are finished; record rejected ideas in the backlog for later.
Merge conflicts between the two developers	Medium	One feature per branch; small, frequent commits; pull-request review before merging to main; shared ownership of design notes.
Existing base code is hard to extend	Medium	Keep original files intact where possible; isolate new conflict / library modules; re-read the base README and preserve the MIT license and attribution.
Insufficient testing time	Medium	Test the conflict and stock-transition path first; lower-priority UI features receive lighter manual checks; use a definition of done per iteration.
Dependency on a shared data state that is not available locally	Low	Use a lightweight shared data store / seeded shared data file; the version-check logic is tested with simulated interleaved requests before any server detail is fixed.

3.6 Iteration Cadence and Definition of Done

- **Before each iteration:** select backlog items from the prioritised list and assign responsibilities; write the acceptance criteria the team will verify.
- **During each iteration:** small commits on feature branches; an issue created for each tracked requirement; a pull request opened before merging.
- **At the end of each iteration:** a short review demonstrating working features against acceptance criteria, updating the risk register and reprioritising the backlog for the next iteration.
- **Definition of done:** the acceptance criteria are met, the affected conflict / stock logic is tested, the branch is merged after review, and the evidence (issue, PR and test record) is linked in the contribution record.

Task 4. Establish and Trial the Collaborative Workflow

4.1 Repository and Workflow Setup

A shared GitHub repository was forked from the MIT-licensed bookmarker base; both members are collaborators and main is protected. The README documents setup (clone; open index.html; no build step) and the manual borrow checks; assessor access is enabled and the submission commit or release tag is recorded on the repository page.

- **Workflow:** backlog item recorded as an Issue; feature branch from main; small commits; pull request reviewed; approved change merged to main.

4.2 Trial Change: Requirement Through Implementation and Testing

One high-priority change was completed end to end: Issue #1, block a borrow at zero stock, linked to BL-2.

- **Requirement:** Issue #1 sets the criterion: borrow blocked at zero stock; stock never negative.
- **Implementation:** Zhou Shuyue created feature/zero-stock-guard and added the stock guard; Bao Nanxi prepared test expectations.
- **Testing:** (a) stock 1 borrows and falls to 0; (b) stock 0 is blocked with a message and unchanged; results recorded as test evidence.
- **Review and merge:** PR #1 was reviewed by Bao Nanxi; an empty-stock comment was addressed; the change was approved and merged; the merge commit is the trial evidence.

4.3 What the Team Learned

Issue-linked branches and peer review prevented uncoordinated main-branch edits and caught an edge case before merge; acceptance criteria made testing objective. Commit counts alone do not show contribution, so Issue, PR, review and test links are recorded per member (Section 4.4); Assignment 2 will add automated checks.

4.4 Individual Contribution Statements

Member	Contributions	Supporting evidence (to link)
Bao Nanxi	Data and version model; Issue #1 criteria; reviewed PR #1 and revision; verified manual checks; authored plan, diagram and report.	Issue #1; PR #1 review; test record; report.
Zhou Shuyue	Repository and README setup; feature/zero-stock-guard implementation; manual checks; opened PR #1 and addressed comments; compiled evidence index.	Repository/README; branch commits; PR #1; test record.

Appendix A: Evidence Index

(excluded from main-report word count)

Evidence item	Location (replace bracket placeholder)	What it shows
Shared repository + README	[repository URL] / README.md	Setup and test instructions; identify submission commit or release tag; assessor access.
Backlog / tracked requirement	Issue #1 [link]; project board [link]	Prioritised BL-1 to BL-6; Issue #1 zero-stock guard acceptance criterion.
Reviewed change	Pull request #1 [link]	Branch diff, reviewer comments, requested revision, approval and merge commit.
Test / check results	[test note or screenshots folder link]	Borrow allowed with stock above zero; blocked at zero; stock never negative.
Contribution record	Section 4.4; commit and PR links [links]	Per-member activities; evidence beyond raw commit counts.

Note: bracketed names, IDs and URLs must be completed from the actual repository before submission; references and this appendix are excluded from the 1,500-word main-report limit.